Smart Student Management System (SSMS)

Documentation

Overview

The **Smart Student Management System (SSMS)** is a Python-based console application for managing student records, academic performance, and generating class-wise reports.

Purpose

- Manage student information and academic records
- Calculate grades and performance metrics
- Generate class-specific reports and rankings
- Export data to text files

Tech Stack

- **Language**: Python 3.11.9
- **Modules**: `datetime`, `os`,`pathlib`
- **Storage**: In-memory dictionary (Hash Map)

Features

1. Add students with roll number, name, class, subjects, and marks
2. Search students by roll number or name (with duplicate handling)
3. Update marks - Add/Update or Remove subjects and marks
4. Display student marksheets with grades and status
5. Generate class-wise reports with statistics and top rankers
6. Export class reports to TXT files with timestamps and displays absolute file path
7. Show top 3 rankers class-wise
8. Delete student records (Admin only)
9. Automatic timestamp tracking (created_on, last_updated)

System Architecture

Data Structure

```
students = {
    'roll_number': {
        'name': str,
        'class': int (1-12),
        'attendance': int,
        'records': {
```

```
            'subjects': list,
            'marks': list
        },
        'created_on': datetime,
        'last_updated': datetime
    }
}
```

Module Organization

```
SSMS/
├── Utility Functions (calculate_percentage, calculate_grade, calculate_status)
├── Validation (validate_roll_number)
├── Student Management (add_student, update_marks,
search_student,display_student)
├── Reports (class_report, show_top_rankers)
├── Export (export_report)
└── Admin (delete_student)
```

Feature Documentation

1. Search Student with Duplicate Handling

Function: `search_student()`

Code:

```python
def search_student():
    search_key = input("Search by Roll No. or Name: ").strip()

    # Direct roll number search
    if search_key in students:
        display_student(search_key)
        return

    # Name search
    matching_students = []
    for roll_number, student in students.items():
        if student['name'].lower() == search_key.lower():
            matching_students.append(roll_number)

    if len(matching_students) > 1:
        # Multiple matches - ask for class
        class_number = int(input("Enter Class (1-12): "))
        class_matching_students = [
            roll for roll in matching_students
            if students[roll]['class'] == class_number
        ]
```

```
        if len(class_matching_students) > 1:
            # List all and ask for roll number
            for roll in class_matching_students:
                print(f"Roll No.: {roll}, Class: {students[roll]['class']}")
            roll_number = input("Enter Roll No. to view details: ").strip()
            display_student(roll_number)
```

---

2. Update Marks with Options

Function: `update_marks()`

Code:

```
def update_marks():
    roll_number = input("Enter Roll Number: ").strip()
    student = students.get(roll_number)

    print("1. Add/Update Marks")
    print("2. Remove Subject")
    choice = input("Enter choice: ").strip()

    if choice == '1':
        # Add or update marks
        new_subjects = [s.strip() for s in input("Enter Subjects: ").split(',')]
        new_marks = list(map(int, input("Enter Marks: ").split(',')))

        for subject, mark in zip(new_subjects, new_marks):
            if subject in subject_list:
                index = subject_list.index(subject)
                old_mark = marks_list[index]
                marks_list[index] = mark
                print(f"Updated {subject}: {old_mark} To {mark}")
            else:
                subject_list.append(subject)
                marks_list.append(mark)
                print(f"Added {subject} with marks {mark}")

    elif choice == '2':
        # Remove subjects
        remove_subjects = [s.strip() for s in input("Subjects to remove:
").split(',')]
        for subject in remove_subjects:
            if subject in subject_list:
                index = subject_list.index(subject)
                subject_list.pop(index)
                marks_list.pop(index)
                print(f"Removed {subject}")
```

```
    student['last_updated'] = datetime.now()
```

---

3. Class Report Generation

Function: `class_report()`

Code:

```
def class_report():
    class_number = int(input("Enter Class (1-12): "))

    # Filter students by class
    class_students = {
        roll: student for roll, student in students.items()
        if student['class'] == class_number
    }

    all_marks = []
    ranked_students = []
    failed_count = 0

    for roll_number, student in class_students.items():
        marks = student['records']['marks']
        all_marks.extend(marks)
        percentage = calculate_percentage(marks)
        status = calculate_status(percentage, student['attendance'])

        ranked_students.append({
            'roll': roll_number,
            'name': student['name'],
            'percentage': percentage
        })

        if status == 'FAIL':
            failed_count += 1

    ranked_students.sort(key=lambda x: x['percentage'], reverse=True)

    # Display statistics
    print(f"Class {class_number} Report:")
    print("Total Students:", len(class_students))
    print("Highest Marks:", max(all_marks))
    print("Lowest Marks:", min(all_marks))
    print("Average Score:", round(sum(all_marks) / len(all_marks), 2))
    print("Failed Students:", failed_count)

    # Top 3 Rankers
```

```
    print(f"\n🏆 Top 3 Rankers - Class {class_number}")
    for rank, student in enumerate(ranked_students[:3], start=1):
        print(f"Rank {rank}")
        print(f"Name: {student['name']}")
        print(f"Roll No: {student['roll']}")
        print(f"Percentage: {round(student['percentage'], 2)}%")
```

---

4. Export Report with Timestamping

Function: export_report()

Code:

```
def export_report():
    class_number = int(input("Enter Class (1-12) to export report: "))

    # Filter class students
    class_students = {
        roll: student for roll, student in students.items()
        if student['class'] == class_number
    }

    # Generate filename with timestamp
    filename =
f"class_{class_number}_report_{datetime.now().strftime('%Y%m%d_%H%M%S')}.txt"
    file_path = os.path.abspath(filename)

    # Calculate statistics and rankings
    all_marks = []
    ranked_students = []
    failed_count = 0

    for roll_number, student in class_students.items():
        marks = student['records']['marks']
        all_marks.extend(marks)
        percentage = calculate_percentage(marks)
        status = calculate_status(percentage, student['attendance'])

        ranked_students.append({
            'roll': roll_number,
            'name': student['name'],
            'percentage': percentage
        })

        if status == 'FAIL':
            failed_count += 1

    ranked_students.sort(key=lambda x: x['percentage'], reverse=True)
```

```
    # Write to file
    with open(filename, 'w', encoding='utf-8') as file:
        file.write(f"Class {class_number} Report\n")
        file.write(f"Generated on: {datetime.now()}\n")
        file.write("=" * 50 + "\n\n")
        file.write(f"Total Students: {len(class_students)}\n")
        file.write(f"Highest Marks: {max(all_marks)}\n")
        file.write(f"Lowest Marks: {min(all_marks)}\n")
        file.write(f"Average Score: {round(sum(all_marks) / len(all_marks), 2)}\n")
        file.write(f"Failed Students: {failed_count}\n\n")

        file.write(f"⏳ Top 3 Rankers - Class {class_number}\n")
        file.write("-" * 50 + "\n")
        for rank, student in enumerate(ranked_students[:3], start=1):
            file.write(f"Rank {rank}\n")
            file.write(f"Name: {student['name']}\n")
            file.write(f"Roll No: {student['roll']}\n")
            file.write(f"Percentage: {round(student['percentage'], 2)}%\n")
            file.write("-" * 50 + "\n")

    print("Report exported successfully")
    print("File location:", file_path)
```

---

5. Top Rankers (Class-wise)

Function: show_top_rankers()

Code:

```
def show_top_rankers():
    class_number = int(input("Enter Class (1-12): "))

    class_students = []
    for roll_number, student in students.items():
        if student['class'] == class_number:
            percentage = calculate_percentage(student['records']['marks'])
            class_students.append({
                'roll': roll_number,
                'name': student['name'],
                'percentage': percentage
            })

    class_students.sort(key=lambda x: x['percentage'], reverse=True)

    print(f"\n⏳ Top 3 Rankers - Class {class_number}")
    for rank, student in enumerate(class_students[:3], start=1):
        print(f"Rank {rank}")
```

```
        print(f"Name: {student['name']}")
        print(f"Roll No: {student['roll']}")
        print(f"Percentage: {round(student['percentage'], 2)}%")
```

---

6. Delete Student (Admin Only)

Function: `delete_student()`

Code:

```
def delete_student():
    username = input("Username: ")
    password = input("Password: ")

    if username == 'admin' and password == 'intern@2025':
        roll_number = input("Enter Roll Number to delete: ").strip()
        if roll_number in students:
            del students[roll_number]
            print("Student deleted")
        else:
            print("Student not found")
    else:
        print("Unauthorized access")
```

Generic Use Case Flow

Application Workflow

1. Application Start
   └─> main() function called
      └─> Menu loop begins (while True)

2. User selects menu option (1-9)

3. Option 1: Add Student
   └─> Input: name, class, roll number, attendance, subjects, marks
   └─> Validate: class range, roll format, duplicates
   └─> Store: Create student dictionary with created_on timestamp
   └─> Return to menu

4. Option 2: Search Student
   └─> Input: roll number or name
   └─> Process: Check roll number or search by name
   └─> Handle: Duplicates → Ask class → Filter → List if needed
   └─> Display: Student marksheet
   └─> Return to menu

5. Option 3: Update Marks
   └─> Input: roll number
   └─> Options: Add/Update or Remove
   └─> Process: Update lists, modify records
   └─> Update: last_updated timestamp
   └─> Return to menu

6. Option 4: Show All Students
   └─> Iterate: All students in dictionary
   └─> Display: Each student marksheet
   └─> Return to menu

7. Option 5: Generate Class Report
   └─> Input: class number
   └─> Filter: Students by class
   └─> Calculate: Statistics, rankings, failed count
   └─> Display: Report with top 3 rankers
   └─> Return to menu

8. Option 6: Export Report
   └─> Input: class number
   └─> Process: Filter, calculate, rank
   └─> Generate: Timestamped filename
   └─> Write: TXT file with report data
   └─> Display: File path
   └─> Return to menu

9. Option 7: Top Rankers
   └─> Input: class number
   └─> Filter: Students by class
   └─> Calculate: Percentages
   └─> Sort: By percentage (descending)
   └─> Display: Top 3 rankers
   └─> Return to menu

10. Option 8: Delete Student
    └─> Authenticate: Username and password
    └─> If authorized: Input roll number → Delete student
    └─> If unauthorized: Display error
    └─> Return to menu

11. Option 9: Exit
    └─> Break loop → Program ends

Core Concepts Applied

1. Variables & Data Types

- String (names, roll numbers)
- Integer (class, marks, attendance)
- Float (percentages)
- List (subjects, marks)
- Dictionary (student records)
- Datetime (timestamps)

2. Operators

- Arithmetic: `+`, `-`, `*`, `/` (calculations)
- Comparison: `>=`, `<=`, `==` (validations, grades)
- Logical: `and`, `or` (status determination)
- Membership: `in`, `not in` (dictionary lookups)

3. Control Flow

- Conditional: `if-elif-else` (grades, validation)
- Loops: `for` (iterations), `while True` (menu)
- Break: Exit conditions

4. Data Structures

- Dictionary: Primary storage (`students = {}`)
- List: Ordered collections (subjects, marks)
- Operations: `.append()`, `.pop()`, `.index()`, `.extend()`

5. Functions

- Function definitions with parameters
- Return values
- Modular organization

6. Error Handling

- `try-except` blocks
- `ValueError` handling
- Edge case handling (empty lists)

7. String Operations

- `.strip()`, `.split()`, `.lower()`, `.isdigit()`
- String slicing

8. File Operations

- `open()` with `'w'` mode and `with` statement
- `os.path.abspath()` for file paths

Special Features

1. Attendance-based PASS/FAIL

What it does: Student can fail even with good marks if attendance < 75%.

Problem solved: Real-world school logic - enforces attendance requirement.

How it works:

```
def calculate_status(percentage, attendance):
    return 'PASS' if percentage >= 40 and attendance >= 75 else 'FAIL'
```

- Requires BOTH: percentage ≥ 40% AND attendance ≥ 75%
- Used in marksheet display, class reports, and exports

---

2. Class-based Roll Number Validation

What it does: Roll format must match class (Class 1 → 0101, 0102; Class 3 → 0301, 0302).

Problem solved: Prevents invalid student entries and ensures data consistency.

How it works:

```
def validate_roll_number(roll, class_number):
    class_part = int(roll[:2])
    if class_part != class_number:
        return False, f"Roll No. must start with class number {class_number:02d}"
```

- Validates format: `CCSSS` (Class + Serial)
- Enforces class-roll number matching during student addition

---

3. Top 3 Rankers (Class-wise)

What it does: Shows Name, Roll number, Class, Percentage, Attendance.

Problem solved: Strong creativity + analytics feature for performance tracking.

How it works: - Available from menu (Option 7: Top Rankers) - Included inside class report (Option 5) - Included in exported reports (Option 6) - Sorts students by percentage (descending), displays top 3

---

4. Admin-only Delete Student

What it does: Secure delete using Username: `admin`, Password: `intern@2025`.

Problem solved: Demonstrates access control and prevents unauthorized deletions.

How it works:

```
def delete_student():
    if input("Username: ") == 'admin' and input("Password: ") == 'intern@2025':
        # Delete operation
    else:
        print("Unauthorized access")
```

---

5. Timestamp Tracking

What it does: Automatically stores Created time and Last updated time.

Problem solved: Audit trail concept for record management.

How it works: - `created_on`: Set when student added (`datetime.now()`) - `last_updated`: Initially `None`, updated on every marks modification - Displayed in marksheet and exported reports

---

6. Export Class Report to TXT

What it does: Generates .txt file with timestamped filename and displays absolute file path.

Problem solved: File handling + reporting for record keeping.

How it works:

```
filename =
f"class_{class_number}_report_{datetime.now().strftime('%Y%m%d_%H%M%S')}.txt"
file_path = os.path.abspath(filename)
print("File location:", file_path)
```

- Format: `class_{number}_report_{YYYYMMDD_HHMMSS}.txt`
- Includes statistics, top rankers, failed count

---

7. Data Integrity Rules

What it does: Prevents negative marks, invalid roll formats, duplicate roll numbers.

Problem solved: Robust input validation ensures data quality.

How it works: - Negative marks check: `if any(mark < 0 for mark in marks_list)` - Roll format validation: `validate_roll_number()` function - Duplicate prevention: `if roll_number in students` - Negative attendance check: `if attendance < 0`

Technical Specifications

Input Validation

- Roll number format: `CCSSS` (Class + Serial, e.g., 0301, 0302)
- Class range: 1-12
- Attendance: 0-100
- Marks: Non-negative integers
- Subject limit: Up to 12 subjects

Performance

- In-memory storage (O(1) dictionary lookup)
- Linear search for name matching
- Built-in Timsort for ranking (O(n log n))

Limitations

- Data lost on program exit (no persistent storage)
- Single-user system
- No data backup mechanism
- No concurrent access handling

Author: Sahil Singh

Document Version: 1.0 **Last Updated**: 01-01-2026 23:40 am